



Project Report: Wumpus World

Name:-

Kashif Hafeez

Roll No:-

F2024376314

Submitted to:-

Instructor's Name:-

Mam Zeenat Tanveer

Section:

A7

1. Class Architecture

The `Wampusgame.py` codebase is structured into four distinct classes with clear separation of concerns:

- **Cell**: Represents a single grid coordinate. Manages boolean flags for hidden entities (Wumpus, pit, gold), sensory cues (stench, breeze, glitter), and exploration status.
- **World**: Manages the 4x4 matrix initialization. Handles procedural hazard/item generation (ensuring the spawn point is safe) and broadcasts sensory clues to orthogonal neighbors.
- **Agent**: Tracks player/AI state, including grid coordinates, running score, survival/victory status, and local memory of visited/safe tiles.
- **Game**: The Pygame-based core engine. Coordinates window setups, input handling, state machine transitions (MENU, PLAYING, GAME_OVER), framerate capping, and UI rendering.

2. AI Logic & Decision-Making

When auto-pilot is active, the agent navigates using localized propositional deduction and a prioritized action heuristic:

Deductive Reasoning

At each turn, the agent analyzes its current cell. If no sensory inputs are present (no breeze, no stench), it logically deduces that all adjacent cells are 100% secure and updates its knowledge base.

Decision Hierarchy

To select its next move, the agent evaluates adjacent cells using a strict checklist:

1. **Grab Gold:** Move immediately to an adjacent cell if gold is detected.
2. **Safe Exploration:** Move to an unvisited, verified-safe neighboring cell.
3. **Safe Backtracking:** Retrace steps through previously explored cells if forward progress is blocked.
4. **Blind Guesswork:** Take a calculated risk on an unverified adjacent cell when no safe options remain.

3. Core Challenges

Challenge	Impact & Context
Limited Deductive Horizon	The AI's logic is local and reactive; it only marks cells safe if the current room is entirely clear. It lacks advanced cross-cell intersection logic, leading to occasional premature blind guesses.
Dynamic Audio Buffering	Audio is synthesized via real-time math calculations (sine/square waves converted to byte arrays) instead of static files. While memory-efficient, this can cause OS-dependent audio driver lag and frame drops.

Challenge	Impact & Context
Asynchronous Interface Dualism	Running a background HTTP web server alongside the main Pygame rendering loop requires strict thread-safe isolation to prevent blocking locks and keep the UI fluid.

4. Key Learning Outcomes

- **Practical Knowledge Representation:** Gained hands-on experience mapping raw sensory percepts (breeze, stench) into an active AI knowledge matrix to drive real-time agent behavior.
- **Finite State Machine (FSM) Mastery:** Implemented a clean state architecture (MENU, PLAYING, GAME_OVER) to isolate input logic from visual rendering layers.
- **Constrained Procedural Generation:** Designed map-generation algorithms that balance randomness with fairness, ensuring the player's starting zone is always safe and solvable.

End 😊